



Консольный файловый менеджер

Курдюков М.М. Королев Е. Е. Симаков Ф. И. Спивак Н. С. Черных С. В.

Задача приложения

Задачи:

- Запуск из текущей директории
- Вывод файлов и папок
- Перемещение курсора стрелками

Ключевые функции :Enter Backspace Delete Space F1 F2 F3 Esc.

Интерфейс и управление

FILE MANAGER

Current: C:\Users\Никита\Desktop

Marked: none

[F] Barony.url
[F] desktop.ini
[D] DS.SolidWorks.2024.SP5.0.Premium-SSQ
[F] FileManager.cpp
[F] FileManager1.cpp
[F] FileManager1.exe
[F] FileManager1.obj
> [F] FileManager_fixed.cpp
[F] FileManager_min.cpp
[F] FileManager_short.cpp

Arrows - move | Enter - open dir | Backspace - parent

Del - delete | Space - mark | F1 - copy | F2 - move | F3 - view | Esc - exit

|

Основная функция

```
int main()
{
    _setmode(_fileno(stdout), _O_U16TEXT);
    _setmode(_fileno(stderr), _O_U16TEXT);

    load();

    while (true)
    {
        draw();

        switch (key())
        {
            case 1: if (cursor > 0) --cursor; break;
            case 2: if (cursor + 1 < (int)items.size()) ++cursor; break;
            case 3: del(); break;
            case 4: copyMarked(); break;
            case 5: moveMarked(); break;
            case 6: viewFile(); break;
            case 7: openDir(); break;
            case 8: goParent(); break;
            case 9: mark(); break;
            case 10: return 0;
        }
    }
}
```

Файловые операции

void openDir()

```
{
    if (items.empty()) return;

    if (items[cursor].isDir)
    {
        currentDir = items[cursor].path;
        load();
    }
}
```

void goParent()

```
{
    currentDir = currentDir.parent_path();
    load();
}
```

void del()

```
{
    if (items.empty()) return;

    fs::remove_all(items[cursor].path);
    load();
}
```

```
void moveMarked()
```

```
{  
    if (!hasMarked) return;
```

```
    fs::path dest = currentDir / marked.filename();
```

```
    fs::rename(marked, dest);  
    marked = dest;
```

```
    load();
```

```
}
```

```
void moveMarked()
```

```
{  
    if (!hasMarked) return;
```

```
    fs::path dest = currentDir / marked.filename();
```

```
    fs::rename(marked, dest);  
    marked = dest;
```

```
    load();
```

```
}
```

```
void mark()
```

```
{
```

```
    if (items.empty()) return;
```

```
    marked = items[cursor].path;  
    hasMarked = true;
```

```
}
```

```
void copyMarked()
```

```
{
```

```
    if (!hasMarked) return;
```

```
    fs::path dest = currentDir / marked.filename();
```

```
    if (fs::is_directory(marked))
```

```
        fs::copy(marked, dest, fs::copy_options::recursive | fs::copy_options::overwrite_existing);
```

```
    else
```

```
        fs::copy_file(marked, dest, fs::copy_options::overwrite_existing);
```

```
    load();
```

```
}
```

```
void viewFile()
```

```
{
```

```
    if (items.empty() || items[cursor].isDir) return;
```

```
    system("cls");
```

```
    ifstream file(items[cursor].path, ios::binary);
```

```
    string text((istreambuf_iterator<char>(file), istreambuf_iterator<char>()));
```

```
    wcout << L"VIEW: " << items[cursor].path.filename().wstring() << L"\n";
```

```
    wcout << L"-----\n";
```

```
    wcout << toWide(text);
```

```
    wcout << L"\n\nPress any key to return...";
```

```
    _getch();
```

```
}
```
